```
"""يولّد data.json (تحليل فني + أساسي) بتوصيات الأسهم الأمريكية.
يعمل تلقائيًا عبر GitHub Actions. ليس نصيحة مالية.
تحليل تعليمي."""
import json, math
from datetime import date, datetime, timezone
import pandas as pd

# عدّل هذه القائمة لتغيير الأسهم المتابَعة
SYMBOLS = ["AAPL", "MSFT", "NVDA", "GOOGL", "AMZN", "META", "TSLA", "AMD",
           "AVGO", "NFLX", "JPM", "V", "COST", "LLY", "CRM", "ORCL"]


def rsi(c, n=14):
    d = c.diff()
    u = d.clip(lower=0).ewm(alpha=1 / n, adjust=False).mean()
    w = (-d.clip(upper=0)).ewm(alpha=1 / n, adjust=False).mean()
    return 100 - 100 / (1 + u / w)


def atr(df, n=14):
    pc = df["Close"].shift(1)
    tr = pd.concat([df["High"] - df["Low"], (df["High"] - pc).abs(),
                    (df["Low"] - pc).abs()], axis=1).max(axis=1)
    return tr.ewm(alpha=1 / n, adjust=False).mean()


def macd_hist(c):
    m = c.ewm(span=12, adjust=False).mean() - c.ewm(span=26, adjust=False).mean()
    return m - m.ewm(span=9, adjust=False).mean()


def get(sym):
    import yfinance as yf
    d = yf.Ticker(sym).history(period="2y", auto_adjust=True)
    return d.dropna() if len(d) else d


def analyze(sym, df, bench):
    if len(df) < 210:
        return None
    c = df["Close"]
    s20, s50, s200 = (c.rolling(n).mean() for n in (20, 50, 200))
    r, a, h = rsi(c), atr(df), macd_hist(c)
    px, A = float(c.iloc[-1]), float(a.iloc[-1])
    ret = px / c.iloc[-126] - 1
```

```
    vr = df["Volume"].iloc[-1] / df["Volume"].rolling(50).mean().iloc[-1]
    dist = px / c.iloc[-252:].max() - 1
    ext = (px - s20.iloc[-1]) / A
    rs = float(r.iloc[-1])
    sc, why = 0, []
    if px > s200.iloc[-1]: sc += 20; why.append("فوق SMA200")
    if s50.iloc[-1] > s200.iloc[-1]: sc += 15; why.append("اتجاه صاعد")
    if px > s50.iloc[-1]: sc += 10
    if h.iloc[-1] > 0 and h.iloc[-1] > h.iloc[-2]: sc += 10; why.append("زخم MACD
    if 45 <= rs <= 65: sc += 15; why.append("RSI صحي")
    elif rs > 75: sc -= 10; why.append("تشبع شراء")
    if ret > bench: sc += 15; why.append("أقوى من السوق")
    if vr > 1.2 and c.iloc[-1] > c.iloc[-2]: sc += 5; why.append("حجم داعم")
    if dist > -0.10: sc += 10; why.append("قرب القمة")
    stop = px - 2 * A
    tgt = px + 2 * (px - stop)
    if sc >= 70 and ext <= 2.5 and rs < 70: rec = "ENTER"
    elif sc >= 55: rec = "PULLBACK" if ext > 2.5 or rs >= 70 else "WATCH"
    else: rec = "AVOID"
    return {"s": sym, "p": round(px, 2), "score": sc, "rec": rec, "rsi": round(rs
            "ret6m": round(float(ret) * 100, 1), "stop": round(stop, 2),
            "tgt": round(tgt, 2), "why": why}


def fundamentals(sym):
    import yfinance as yf
    t = yf.Ticker(sym)
    try: i = t.info or {}
    except Exception: i = {}
    fpe, peg = i.get("forwardPE"), i.get("pegRatio") or i.get("trailingPegRatio")
    rg, eg, pm = i.get("revenueGrowth"), i.get("earningsGrowth"), i.get("profitMa
    days = None
    try:
        ed = (t.calendar or {}).get("Earnings Date")
        if ed:
            d = ed[0] if isinstance(ed, (list, tuple)) else ed
            d = d.date() if isinstance(d, datetime) else d
            days = (d - date.today()).days
    except Exception:
        pass
    if all(v is None for v in (fpe, rg, eg, pm)):
        return None, [], days
    f, n = 0, []
    if rg is not None and rg > 0.10: f += 5; n.append(f"نمو إيراد {rg:.0%}")
    if eg is not None and eg > 0.10: f += 5; n.append(f"نمو أرباح {eg:.0%}")
    if pm is not None and pm > 0.15: f += 5; n.append(f"هامش {pm:.0%}")
    if fpe is not None:
```

```
        if 0 < fpe < 30: f += 5; n.append(f"P/E {fpe:.0f}")
        elif fpe > 60: f -= 5; n.append(f"P/E مرتفع {fpe:.0f}")
    if peg is not None and 0 < peg < 2: f += 5; n.append(f"PEG {peg:.1f}")
    return max(0, f), n, days


def main():
    spy = get("SPY")
    bench = spy["Close"].iloc[-1] / spy["Close"].iloc[-126] - 1
    rows = []
    for s in SYMBOLS:
        try:
            r = analyze(s, get(s), bench)
            if not r:
                continue
            f, notes, days = fundamentals(s)
            r["total"] = round(r["score"] if f is None else 0.7 * r["score"] + 0.
            r["earn_days"] = days
            if r["rec"] == "ENTER":
                if r["total"] < 65: r["rec"] = "WATCH"
                elif days is not None and 0 <= days <= 7: r["rec"] = "EARNINGS"
            r["why"] += notes
            rows.append(r)
        except Exception as e:
            print(s, "error", e)
    rows.sort(key=lambda r: r["total"], reverse=True)
    out = {"updated": datetime.now(timezone.utc).isoformat(timespec="minutes"), "
    with open("data.json", "w", encoding="utf-8") as fh:
        json.dump(out, fh, ensure_ascii=False)
    print(f"{len(rows)} سهم محدّث")


if __name__ == "__main__":
    main()
```